BARQ on Kubernetes: Ansible-Automated Cluster on Azure

Overview and Result

A two-node Kubernetes v1.31.14 cluster was built on Azure by one Ansible playbook, and the BARQ application now runs on it: three app replicas behind NGINX, with PostgreSQL and Redis on persistent volumes. Three requests from three different clients were answered by three different replicas (Figures 1 to 3).

This document continues the earlier manual kubeadm build on VMware. It starts at Azure provisioning and ends with the running application; the terminal outputs and screenshots shown are from that run.

Component	Final state
Cluster	`master-node` (control plane) and `worker-1`, both Ready, v1.31.14
Pod network	Calico v3.27.0 in VXLAN mode, Pod CIDR `10.244.0.0/16`
Application	namespace `barq`: 3 × `app`, 1 × `nginx`, `postgres-0`, `redis-0`, all Running with 0 restarts
Storage	2 PersistentVolumeClaims Bound on `local-path` (PostgreSQL 2 Gi, Redis 1 Gi)
Access	NGINX Service of type NodePort, port 30080 on the worker's public IP

Environment

The cluster runs on two Azure VMs in one subnet; Ansible, kubectl, Docker and Trivy run from an Ubuntu WSL2 machine outside Azure.

Item	Value
Region	Switzerland North

Item	Value
VM size	Standard_B2als_v2 (2 vCPU, 4 GiB RAM)
Nodes	`master-node` (control plane), `worker-1`
Admin user	`azureuser`, SSH key authentication
Private addresses	`master-node` 172.16.0.4, `worker-1` 172.16.0.5
Worker public IP	20.199.168.151
Container runtime	containerd, `SystemdCgroup = true`
Kubernetes	v1.31 repository, installed v1.31.14 (kubeadm, kubelet, kubectl, packages held)
CNI	Calico v3.27.0, VXLAN mode
Pod CIDR	`10.244.0.0/16`
Storage	local-path-provisioner v0.0.37
Image registry	GitHub Container Registry, `ghcr.io/mazenhassan20/barq-app`

Azure Provisioning

Both VMs sit in one resource group, VNet and subnet, so they reach each other over private addresses; only SSH, the Kubernetes API and one application port are open to the internet.

VM settings

Setting	Value
Names	`master-node`, `worker-1`
Region and size	Switzerland North, Standard_B2als_v2
Image	Ubuntu Server LTS, x64
Authentication	SSH public key, one key pair for both VMs, user `azureuser`

Setting	Value
Network	Same VNet and subnet for both VMs
Public IP	Standard SKU, static

The public IP must be static because the master's public IP is added to the API server certificate. If it changed, `kubectl` from the WSL machine would be rejected until the certificate was regenerated.

Network security group

Purpose	Port	Protocol	Source
SSH for Ansible	22	TCP	Admin machine's public IP
Kubernetes API for `kubectl`	6443	TCP	Admin machine's public IP
BARQ through the NGINX NodePort	30080	TCP	Admin machine's public IP (wider only for a demonstration)

Traffic between the two nodes (VXLAN on UDP 4789, kubelet on 10250, etcd on 2379 to 2380, and Pod traffic) stays inside the VNet. Azure's default AllowVnetInBound rule covers it, so no extra rules were added.

Ansible Automation

One playbook, `k8s-setup.yml`, turns two freshly created Ubuntu VMs into a working cluster. It is written to be re-runnable: `kubeadm init` and `kubeadm join` are skipped once they have completed. The full file is in Appendix A.

Inventory

```
[master]
master-node ansible_host=20.199.169.82 private_ip=172.16.0.4

[workers]
worker-1 ansible_host=20.199.168.151

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/k8s_azure
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=accept-new'
```

`ansible_host` is the public address Ansible connects to. `private_ip` is the address the API server advertises to the nodes.

Plays

#	Play	Hosts	What it does
1	Common setup	all	Sets the hostname; turns swap off and comments the swap line in `/etc/fstab`; loads `overlay` and `br_netfilter`; sets three sysctl values; installs containerd, `conntrack` and the apt prerequisites; enables `SystemdCgroup`; adds the Kubernetes v1.31 apt repository; installs kubelet, kubeadm and kubectl and holds them
2	Control plane	master	Runs `kubeadm init`; copies `admin.conf` to `~/.kube/config`; downloads, patches and applies the Calico manifest; creates the join command and stores it as a fact
3	Worker	workers	Runs the join command stored by the master
4	Verify	master	Waits up to 300 seconds for all nodes to be Ready, then prints `kubectl get nodes -o wide`

Design decisions

Decision	Reason
Calico in VXLAN mode: backend `vxlan`, IP-in-IP `Never`, VXLAN `Always`, BIRD probes removed	Azure drops IP-in-IP (IP protocol 4), which is Calico's default, so Pods on different nodes could not reach each other
`--pod-network-cidr=10.244.0.0/16`, also written into the Calico pool	Keeps Pod addresses apart from the 172.16.0.x node subnet, and kubeadm and Calico agree on the range
`--apiserver-advertise-address` set to the master's private IP	The nodes reach the API server over the VNet
`--apiserver-cert-extra-sans` set to the master's public IP	`kubectl` from the WSL machine validates the API server certificate
Swap line commented with the `replace` module	The `sed` one-liner from the VMware build left the `/swap.img` entry active and swap returned after a reboot
`creates:` on `kubeadm init` and `kubeadm join`; containerd config generated only if `SystemdCgroup = true` is missing	A second run does not re-initialise the cluster or overwrite the runtime config
Join command from `kubeadm token create --print-join-command`, passed to workers through `hostvars`	No token is copied by hand
`k8s_version` variable set to 1.31, packages held	v1.30 is out of support, and a hold prevents unplanned upgrades
`conntrack` installed on every node	kube-proxy needs it, and `kubeadm init` failed its preflight check without it in the VMware build

Running it

From the directory that holds both files:

```
ansible all -i hosts.ini -m ping
ansible-playbook -i hosts.ini k8s-setup.yml --syntax-check
ansible-playbook -i hosts.ini k8s-setup.yml
```

The ping must answer `pong` from both hosts before the playbook is run. After the run, `kubectl get nodes` showed both nodes Ready, the control plane aged 6m50s and the worker 5m57s.

Cluster Verification

Both nodes are Ready, all 11 system Pods are Running, and a Pod on one node reaches a Pod on the other with no packet loss.

Nodes and system Pods

```
NAME          STATUS   ROLES           AGE     VERSION
master-node   Ready    control-plane   6m50s   v1.31.14
worker-1      Ready    <none>          5m57s   v1.31.14
```

System component	Pods	Node	Address
etcd, kube-apiserver, kube-controller-manager, kube-scheduler	4	`master-node`	172.16.0.4
calico-node, kube-proxy	2 each	one per node	172.16.0.4 and 172.16.0.5
coredns	2	`master-node`	10.244.77.129 and 10.244.77.130
calico-kube-controllers	1	`master-node`	10.244.77.131

Pods on each node draw addresses from a separate block of the Pod CIDR: 10.244.77.x on `master-node` and 10.244.226.x on `worker-1`.

Cross-node Pod connectivity

The test placed one Pod on each node and pinged across.

1. `test-pod-1` (nginx) was created and scheduled on `worker-1`, address 10.244.226.65.
2. `test-pod-2` (busybox:1.36) was pinned to `master-node` with a node selector. It stayed Pending, and `kubectl describe pod` explained why: `1 node(s) had untolerated taint {node-role.kubernetes.io/control-plane: }`.

3. The Pod was recreated with a toleration for that taint and started on `master-node`, address 10.244.77.132.

```
kubectl run test-pod-2 --image=busybox:1.36 --restart=Never \
  --overrides='{"spec":{"nodeSelector":{"kubernetes.io/hostname":"master-
node"},"tolerations":[{"key":"node-role.kubernetes.io/control-
plane","operator":"Exists","effect":"NoSchedule"}]}}' \
  -- sleep 3600
kubectl exec -it test-pod-2 -- ping -c 4 10.244.226.65
```

```
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.482/0.787/1.145 ms
```

This confirms the VXLAN Pod network works between nodes on Azure.

DNS

The `kube-dns` Service has ClusterIP 10.96.0.10 and is backed by the two CoreDNS Pods. From `test-pod-2`, `nslookup kubernetes.default.svc.cluster.local` returned 10.96.0.1. The short name `kubernetes.default` returned NXDOMAIN. That matches a known limitation of `nslookup` in newer busybox images, which is why Kubernetes documentation uses busybox 1.28 for DNS checks; it was not investigated further.

Restart resilience

After the VMs were stopped and started again (cluster age 24 hours), all 11 `kube-system` Pods returned to Running with 1 to 2 restarts each (Figure 2, top).

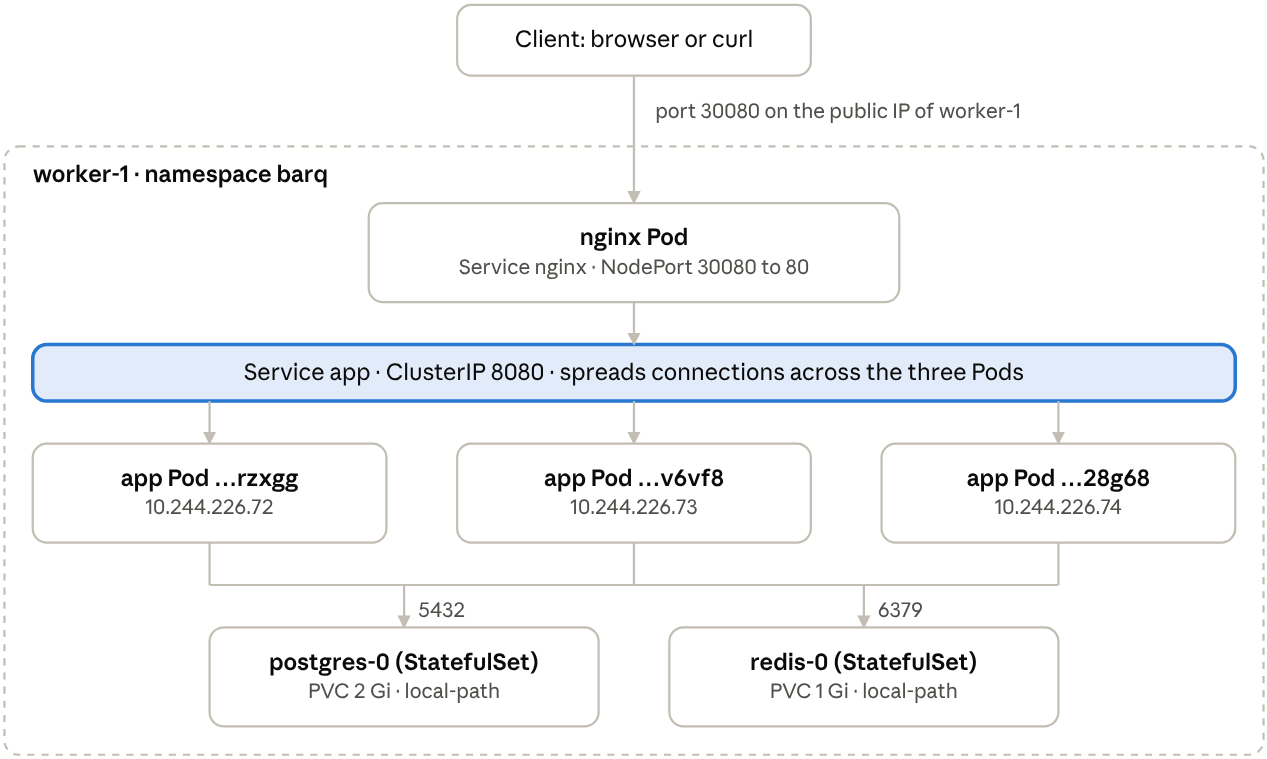
Application and Kubernetes Design

The BARQ Docker Compose stack maps onto Kubernetes objects one to one, except that the three app containers become a single Deployment with three replicas. All objects live in the `barq` namespace, in the `k8s/` folder of the repository.

Compose element	Kubernetes object	Key settings
`postgres`	StatefulSet `postgres`, ClusterIP Service `postgres` on 5432	`postgres:16-alpine` pinned by digest; user `barq_app`, database `barq_tasks`; password from Secret `barq-env`; 2 Gi PVC on `local-path`; `init.sql` from ConfigMap `postgres-init`; `pg_isready` probes; limits 1 CPU and 512 Mi
`redis`	StatefulSet `redis`, ClusterIP Service `redis` on 6379	`redis:7.4-alpine` pinned by digest; `redis-server --save 60 1 --appendonly yes`; 1 Gi PVC; `redis-cli ping` probes; limits 250m CPU and 256 Mi
`app-01`, `app-02`, `app-03`	Deployment `app` with 3 replicas, ClusterIP Service `app` on 8080	One image, `ghcr.io/mazenhassan20/barq-app:v1`, built from the repository Dockerfile; environment from Secret `barq-env`; `INSTANCE_ID` taken from the Pod name; `/health` readiness and liveness probes; non-root user 10001, all capabilities dropped; limits 500m CPU and 256 Mi
`nginx`	Deployment `nginx`, NodePort Service 80 to 30080	`nginx:1.28-alpine` pinned by digest; configuration from ConfigMap `nginx-conf` (file `nginx.k8s.conf`); `/health` probes
`frontend` and `backend` networks	Three NetworkPolicies	Only `app` Pods may reach Postgres (5432) and Redis (6379); only `nginx` may reach `app` (8080)
`.env` file	Secret `barq-env`	Created by `deploy.sh` with `--from-env-file`

The `app` Service load-balances across all three replicas, so NGINX proxies to `app:8080` instead of to container names. The NetworkPolicies are enforced by Calico and restrict ingress only; unlike the Compose `internal` network, they do not block outbound traffic from these Pods.

Every request enters through one NodePort and reaches one of three app replicas



NetworkPolicies allow only app Pods to reach Postgres and Redis, and only nginx to reach app.

request path · 6 Pods on worker-1

Requests reach NGINX through the NodePort, the `app` Service picks one of the three app Pods for each connection, and the app talks to PostgreSQL and Redis. All six Pods run on `worker-1`.

Prerequisites for the Deployment

Three things had to exist before the manifests could run: a storage class, an image the nodes can pull, and a `kubectl` connection from the WSL machine.

1. Storage class

A kubeadm cluster has no storage provisioner, so the PersistentVolumeClaims of Postgres and Redis would stay Pending. Rancher's local-path-provisioner was installed on the master:

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-
provisioner/v0.0.37/deploy/local-path-storage.yaml
kubectl get storageclass
```

```
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer
false
```

The class is not marked as default, so the manifests request it by name (`storageClassName: local-path`). Volumes are directories on the disk of the node where the Pod runs, and `WaitForFirstConsumer` delays binding until the Pod is scheduled.

2. Application image in a registry

The nodes cannot see images built on the WSL machine, so the image was built from the repository Dockerfile and pushed to GitHub Container Registry. The package was set to public so the nodes can pull it without credentials.

```
docker build -t ghcr.io/mazenhassan20/barq-app:v1 .
echo $GHCR_PAT | docker login ghcr.io -u mazenhassan20 --password-stdin
docker push ghcr.io/mazenhassan20/barq-app:v1
```

`GHCR_PAT` is a personal access token with the `write:packages` scope. It lives only in the shell environment.

3. `kubectl` from the WSL machine

The kubeconfig was copied from the master and its `server:` line pointed at the master's public address, which is why the public IP is in the API server certificate:

```
scp -i ~/.ssh/k8s_azure azureuser@<MASTER_PUBLIC_IP>:~/.kube/config
~/.kube/azure-config
sed -i 's#server: .*#server: https://<MASTER_PUBLIC_IP>:6443#' ~/.kube/azure-
config
export KUBECONFIG=~/.kube/azure-config
kubectl get nodes
```

This needs port 6443 open to the WSL machine's public IP in the network security group.

4. Application settings

A `.env` file next to the Compose file holds the application settings, including `POSTGRES_PASSWORD`. `deploy.sh` turns it into the Secret `barq-env`. The file is excluded from Git (see Repository Layout and Secrets Handling).

Deployment and Evidence

`deploy.sh`, run from the repository root on the WSL machine, brought the whole application up and every rollout completed without errors. The script can be re-run: Secrets and ConfigMaps are generated with `--dry-run=client` and applied, so existing ones are updated rather than rejected.

What `deploy.sh` does

1. Applies the `barq` namespace.
2. Creates the Secret `barq-env` from `.env`.
3. Creates the ConfigMaps `postgres-init` (from `database/init.sql`) and `nginx-conf` (from `k8s/nginx.k8s.conf`).
4. Applies Postgres and Redis, then waits up to 180 seconds for both StatefulSets.
5. Applies the app, NGINX and the NetworkPolicies, then waits for both Deployments.
6. Prints the Pods, Services and PersistentVolumeClaims.

Evidence

Check	Result	Figure
Rollout	`app` reached 3 of 3 available replicas; `nginx` became available	1
Pods	6 Running with 0 restarts, all on `worker-1` (10.244.226.70 to 10.244.226.75)	1, 2
Services	`app` ClusterIP 10.111.119.165:8080; `nginx` NodePort 80:30080; `postgres` 10.110.226.121:5432; `redis` 10.104.207.108:6379	1
Persistent storage	`postgres-data-postgres-0` (2 Gi) and `redis-data-redis-0` (1 Gi) Bound on `local-path`	1

Check	Result	Figure
Load balancing	Three clients, three different replicas: WSL `curl` answered by `...28g68`, master `curl` by `...rzxgg`, browser by `...v6vf8`	1, 2, 3
Cluster restart	`kube-system` Pods Running after the VMs were restarted	2

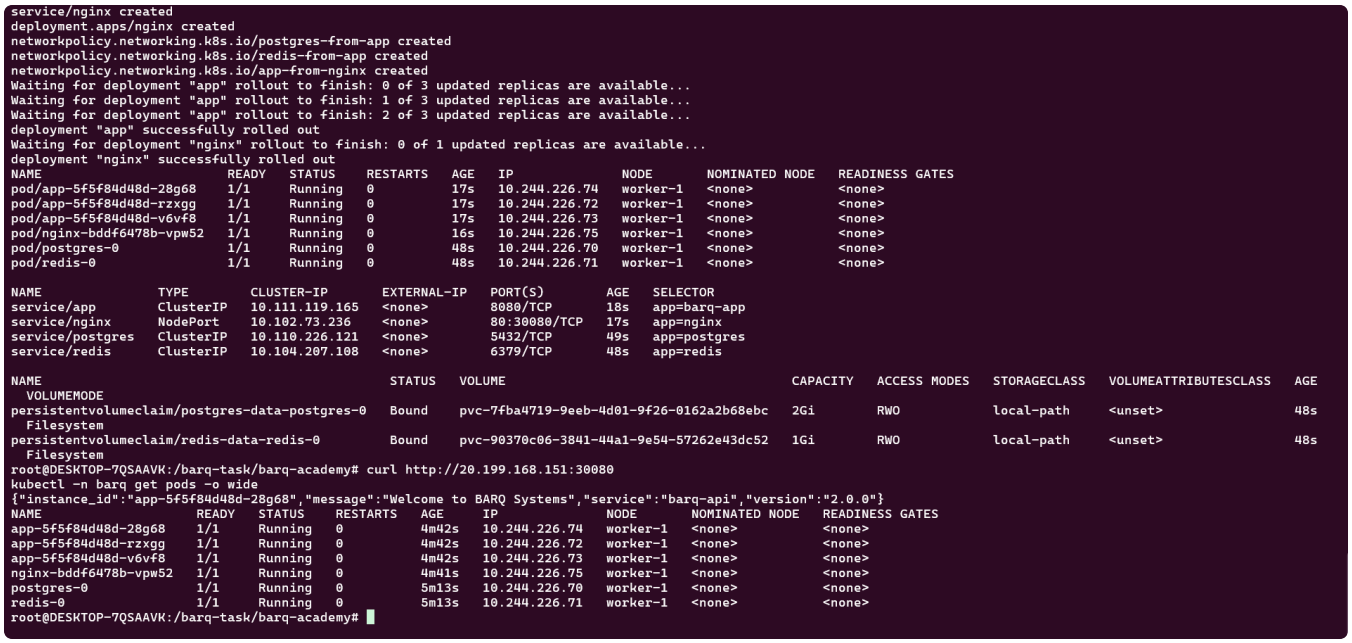


Figure 1. Output of `./deploy.sh` and a `curl` from the WSL machine: both Deployments rolled out, six Pods are Running on `worker-1`, the `nginx` Service is a NodePort on 80:30080, both PVCs are Bound, and the request was answered by `app-5f5f84d48d-28g68`.

```
kube-system   calico-kube-controllers-868cbf9cc-zg545   1/1     Running   2 (26m ago)   24h
kube-system   calico-node-d7rjg                         1/1     Running   2 (26m ago)   24h
kube-system   calico-node-znxp6                         1/1     Running   1 (52m ago)   24h
kube-system   coredns-7c65d6cfc9-8lclg                  1/1     Running   2 (26m ago)   24h
kube-system   coredns-7c65d6cfc9-w2dzj                  1/1     Running   2 (26m ago)   24h
kube-system   etcd-master-node                          1/1     Running   2 (26m ago)   24h
kube-system   kube-apiserver-master-node                1/1     Running   2 (26m ago)   24h
kube-system   kube-controller-manager-master-node       1/1     Running   2 (26m ago)   24h
kube-system   kube-proxy-5gkq7                          1/1     Running   1 (52m ago)   24h
kube-system   kube-proxy-w95pj                          1/1     Running   2 (26m ago)   24h
kube-system   kube-scheduler-master-node                1/1     Running   2 (26m ago)   24h
azureuser@master-node:~$ kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.37/deploy/local-path
namespace/local-path-storage created
serviceaccount/local-path-provisioner-service-account created
role.rbac.authorization.k8s.io/local-path-provisioner-role created
clusterrole.rbac.authorization.k8s.io/local-path-provisioner-role created
rolebinding.rbac.authorization.k8s.io/local-path-provisioner-bind created
clusterrolebinding.rbac.authorization.k8s.io/local-path-provisioner-bind created
deployment.apps/local-path-provisioner created
storageclass.storage.k8s.io/local-path created
configmap/local-path-config created
azureuser@master-node:~$ kubectl get storageclass
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer   false                  8s
azureuser@master-node:~$ kubectl get storageclass
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer   false                  19s
azureuser@master-node:~$ curl http://20.199.168.151:30080
kubectl -n barq get pods -o wide
{"instance_id":"app-5f5f84d48d-rzxgg","message":"Welcome to BARQ Systems","service":"barq-api","version":"2.0.0"}
NAME                   READY   STATUS    RESTARTS   AGE     IP              NODE       NOMINATED NODE   READINESS GATES
app-5f5f84d48d-28g68   1/1     Running   0          5m24s   10.244.226.74   worker-1   <none>           <none>
app-5f5f84d48d-rzxgg   1/1     Running   0          5m24s   10.244.226.72   worker-1   <none>           <none>
app-5f5f84d48d-v6vf8   1/1     Running   0          5m24s   10.244.226.73   worker-1   <none>           <none>
nginx-bddf6478b-vpw52  1/1     Running   0          5m23s   10.244.226.75   worker-1   <none>           <none>
postgres-0             1/1     Running   0          5m55s   10.244.226.70   worker-1   <none>           <none>
redis-0                1/1     Running   0          5m55s   10.244.226.71   worker-1   <none>           <none>
```

Figure 2. The master node. Top: `kube-system` Pods Running after the VM restart. Middle: local-path-provisioner created and the `local-path` StorageClass listed. Bottom: a `curl` to the NodePort answered by `app-5f5f84d48d-rzxgg`, and the six `barq` Pods Running.

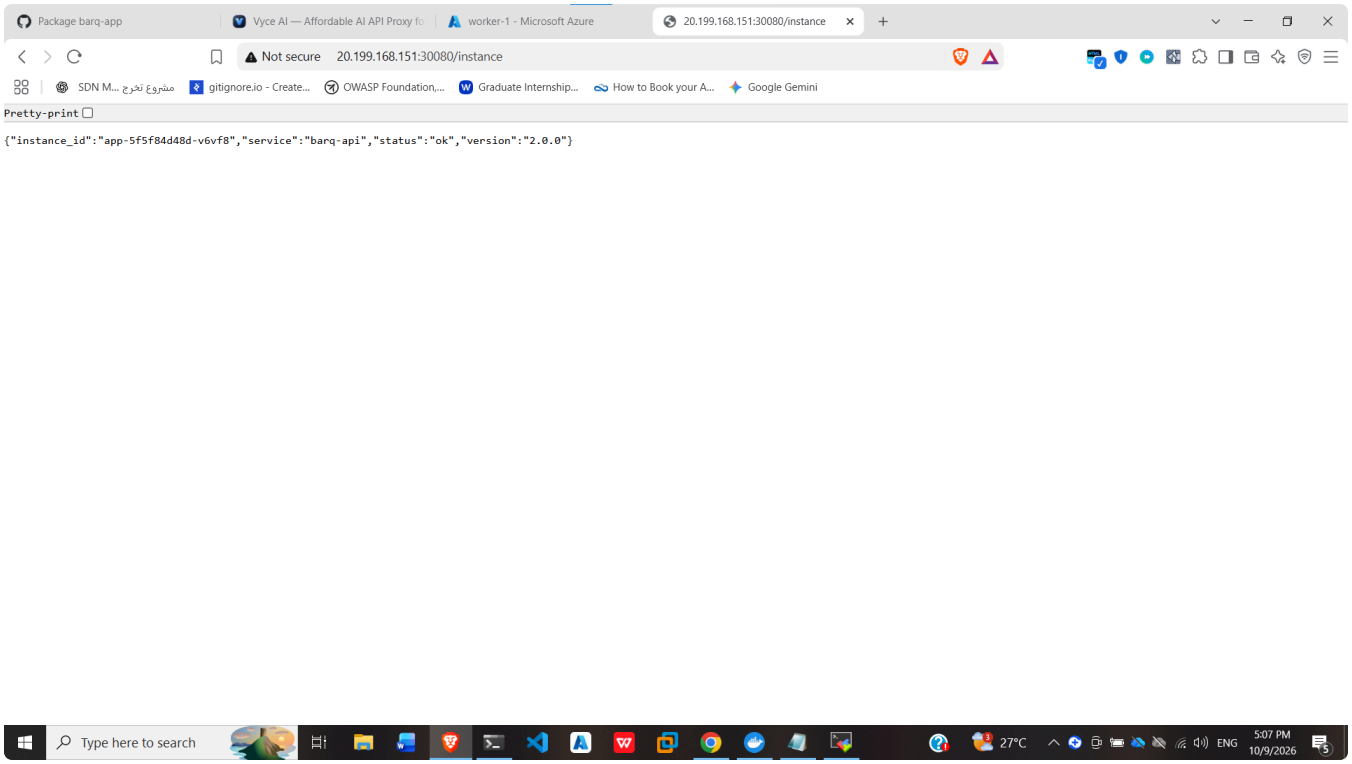


Figure 3. A browser request to `/instance` on the worker's public address, answered by `app-5f5f84d48d-v6vf8` from service `barq-api`, version 2.0.0.

Repeating the check

```
curl http://<WORKER_PUBLIC_IP>:30080
curl http://<WORKER_PUBLIC_IP>:30080/instance
kubectl -n barq get pods -o wide
```

The root path returns the welcome message with the `instance_id` of the replica that answered; `/instance` returns the `instance_id` and `status`. In this run the three clients received three different `instance_id` values.

Known Limitations

The deployment is a working baseline, not a production setup. These are its limits as built:

Limitation	Consequence
One control plane and one worker	The control-plane taint keeps workloads off the master, so all six Pods run on `worker-1`; if that node is lost, the application is down
Node-local volumes	`local-path` volumes are directories on `worker-1`'s disk with reclaim policy `Delete`; deleting a PVC deletes its data, and no backup of the PostgreSQL or Redis data is configured in the cluster
Manual image tag	`barq-app:v1` is built and pushed by hand, so the manifests do not show which commit is running
Plain Kubernetes Secret	`barq-env` is base64-encoded, not encrypted at rest (kubeadm leaves etcd encryption off), and it is created from a local file outside Git
HTTP on a NodePort	The application is served over plain HTTP on port 30080 of the worker's public IP; only the network security group limits who can reach it
NGINX configuration	`nginx.k8s.conf` is a baseline reverse proxy written for the cluster; headers, limits and logging from the original `nginx/nginx.conf` have to be compared and carried over

Limitation	Consequence
NetworkPolicies not tested	They were applied and accepted by the cluster, but no blocked connection was demonstrated
Access depends on one source IP	Ports 22 and 6443 accept only the admin machine's public IP; if that IP changes, Ansible and `kubectl` lose access until the NSG rule is updated

Repository Layout and Secrets Handling

Everything needed to reproduce the build is committed to the repository; anything that grants access is not.

```
barq-academy/
  ansible/
    k8s-setup.yml
    hosts.ini.example
  k8s/
    00-namespace.yaml
    10-postgres.yaml
    20-redis.yaml
    30-app.yaml
    40-nginx.yaml
    50-networkpolicies.yaml
    nginx.k8s.conf
  deploy.sh
  docs/
  .env.example
  .gitignore
```

Item	Committed	Reason
`ansible/k8s-setup.yml`	Yes	Holds no secrets; host details come from the inventory
`ansible/hosts.ini.example`	Yes	Placeholders only
`ansible/hosts.ini`	No	Contains real public IPs
`k8s/` and `deploy.sh`	Yes	Secrets are created at deploy time, not stored in the manifests

Item	Committed	Reason
`docs/` with screenshots	Yes, after review	Screenshots show the worker's public IP; check that none shows a token or key
`.env`	No	Holds `POSTGRES_PASSWORD` and the application settings; `.env.example` carries the variable names without values
`~/.kube/config`, `~/.kube/azure-config`	No	Cluster administrator credentials
`~/.ssh/k8s_azure`	No	SSH private key for both VMs
GHCR access token	No	Stays in the shell environment

Entries for `.gitignore`:

```
.env
hosts.ini
*.pem
k8s_azure*
azure-config
kubeconfig*
```

`trivy fs --scanners secret .` run on the repository on 2026-10-08 reported no findings. It should be run again before the first push of these files.

Appendix A: k8s-setup.yml

The playbook in full. It needs the `community.general` and `ansible.posix` collections, which ship with the full `ansible` package. The inventory template is in the Ansible Automation section.

```
---

- name: Common setup (all nodes)
  hosts: all
  become: true
  vars:
    k8s_version: "1.31"
```

```
  tasks:
    - name: Set hostname to the inventory name
      ansible.builtin.hostname:
        name: "{{ inventory_hostname }}"

    - name: Disable swap for the running system
      ansible.builtin.command: swapoff -a
      when: ansible_swaptotal_mb | int > 0

    - name: Remove swap entries from /etc/fstab
      ansible.builtin.replace:
        path: /etc/fstab
        regexp: '^([^#].*\sswap\s.*)$'
        replace: '# \1'

    - name: Configure kernel modules to load at boot
      ansible.builtin.copy:
        dest: /etc/modules-load.d/k8s.conf
        content: |
          overlay
          br_netfilter
        mode: "0644"

    - name: Load kernel modules now
      community.general.modprobe:
        name: "{{ item }}"
        state: present
      loop:
        - overlay
        - br_netfilter

    - name: Apply sysctl settings for Kubernetes networking
      ansible.posix.sysctl:
        name: "{{ item.name }}"
        value: "{{ item.value }}"
        sysctl_file: /etc/sysctl.d/k8s.conf
        sysctl_set: true
        reload: true
        state: present
      loop:
        - { name: net.bridge.bridge-nf-call-iptables, value: "1" }
        - { name: net.bridge.bridge-nf-call-ip6tables, value: "1" }
        - { name: net.ipv4.ip_forward, value: "1" }
```

```
    - name: Install prerequisite packages and containerd
      ansible.builtin.apt:
        name:
          - containerd
          - apt-transport-https
          - ca-certificates
          - curl
          - gpg
          - conntrack
        state: present
        update_cache: true
        cache_valid_time: 3600
        lock_timeout: 180

    - name: Ensure /etc/containerd exists
      ansible.builtin.file:
        path: /etc/containerd
        state: directory
        mode: "0755"

    - name: Check whether containerd already uses the systemd cgroup driver
      ansible.builtin.command: grep -qs "SystemdCgroup = true"
/etc/containerd/config.toml
      register: cgroup_check
      failed_when: false
      changed_when: false

    - name: Generate default containerd config
      ansible.builtin.shell: containerd config default >
/etc/containerd/config.toml
      args:
        executable: /bin/bash
      when: cgroup_check.rc != 0

    - name: Enable SystemdCgroup in containerd config
      ansible.builtin.replace:
        path: /etc/containerd/config.toml
        regexp: 'SystemdCgroup = false'
        replace: 'SystemdCgroup = true'
      notify: Restart containerd

    - name: Enable and start containerd
```

```
      ansible.builtin.systemd:
        name: containerd
        enabled: true
        state: started

    - name: Create apt keyrings directory
      ansible.builtin.file:
        path: /etc/apt/keyrings
        state: directory
        mode: "0755"

    - name: Download and install the Kubernetes apt signing key
      ansible.builtin.shell: >
        set -o pipefail &&
        curl -fsSL https://pkgs.k8s.io/core:/stable:/v{{ k8s_version
}}/deb/Release.key
        | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
      args:
        executable: /bin/bash
        creates: /etc/apt/keyrings/kubernetes-apt-keyring.gpg

    - name: Add the Kubernetes apt repository
      ansible.builtin.apt_repository:
        repo: "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v{{ k8s_version }}/deb/ /"
        filename: kubernetes
        state: present
        update_cache: true

    - name: Install kubelet, kubeadm and kubectl
      ansible.builtin.apt:
        name:
          - kubelet
          - kubeadm
          - kubectl
        state: present
        lock_timeout: 180

    - name: Hold Kubernetes packages at the installed version
      ansible.builtin.dpkg_selections:
        name: "{{ item }}"
        selection: hold
      loop:
```

```
        - kubelet
        - kubeadm
        - kubectl

    - name: Enable kubelet
      ansible.builtin.systemd:
        name: kubelet
        enabled: true
        state: started

    - name: Apply pending containerd restart before continuing
      ansible.builtin.meta: flush_handlers

  handlers:
    - name: Restart containerd
      ansible.builtin.systemd:
        name: containerd
        state: restarted

- name: Control plane setup
  hosts: master
  become: true
  vars:
    pod_cidr: "10.244.0.0/16"
    calico_version: "v3.27.0"
    kubeconfig: /etc/kubernetes/admin.conf
  tasks:
    - name: Initialize the cluster
      ansible.builtin.command: >
        kubeadm init
        --apiserver-advertise-address={{ private_ip }}
        --apiserver-cert-extra-sans={{ ansible_host }}
        --pod-network-cidr={{ pod_cidr }}
      args:
        creates: /etc/kubernetes/admin.conf

    - name: Create .kube directory for the login user
      ansible.builtin.file:
        path: "/home/{{ ansible_user }}/.kube"
        state: directory
        owner: "{{ ansible_user }}"
        group: "{{ ansible_user }}"
        mode: "0755"
```

```
    - name: Copy admin.conf for the login user
      ansible.builtin.copy:
        src: /etc/kubernetes/admin.conf
        dest: "/home/{{ ansible_user }}/.kube/config"
        remote_src: true
        owner: "{{ ansible_user }}"
        group: "{{ ansible_user }}"
        mode: "0600"

    - name: Download the Calico manifest
      ansible.builtin.get_url:
        url: "https://raw.githubusercontent.com/projectcalico/calico/{{
calico_version }}/manifests/calico.yaml"
        dest: /tmp/calico.yaml
        mode: "0644"
        force: false

    # Azure drops IP-in-IP (protocol 4), so Calico must run in VXLAN mode.
    - name: Switch Calico backend to vxlan
      ansible.builtin.replace:
        path: /tmp/calico.yaml
        regexp: 'calico_backend: "bird"'
        replace: 'calico_backend: "vxlan"'

    - name: Disable IP-in-IP on the default pool
      ansible.builtin.replace:
        path: /tmp/calico.yaml
        regexp: '(- name: CALICO_IPV4POOL_IPIP\n\s+value: )"Always"'
        replace: '\1"Never"'

    - name: Enable VXLAN on the default pool
      ansible.builtin.replace:
        path: /tmp/calico.yaml
        regexp: '(- name: CALICO_IPV4POOL_VXLAN\n\s+value: )"Never"'
        replace: '\1"Always"'

    - name: Remove BIRD health checks (BIRD is not used with vxlan)
      ansible.builtin.replace:
        path: /tmp/calico.yaml
        regexp: '^[ ]*- -bird-(live|ready)\n'
        replace: ''
```

```
    - name: Uncomment CALICO_IPV4POOL_CIDR name
      ansible.builtin.replace:
        path: /tmp/calico.yaml
        regexp: '^([ ]*)# - name: CALICO_IPV4POOL_CIDR$'
        replace: '\1- name: CALICO_IPV4POOL_CIDR'

    - name: Set CALICO_IPV4POOL_CIDR value to the pod CIDR
      ansible.builtin.replace:
        path: /tmp/calico.yaml
        regexp: '^([ ]*)#[ ]+value: "192\.168\.0\.0/16"$'
        replace: '\1  value: "{{ pod_cidr }}"'

    - name: Apply Calico
      ansible.builtin.command: kubectl apply -f /tmp/calico.yaml
      environment:
        KUBECONFIG: "{{ kubeconfig }}"
      register: calico_apply
      changed_when: "'created' in calico_apply.stdout or 'configured' in
calico_apply.stdout"

    - name: Generate the worker join command
      ansible.builtin.command: kubeadm token create --print-join-command
      register: join_cmd
      changed_when: false

    - name: Store the join command as a fact
      ansible.builtin.set_fact:
        join_command: "{{ join_cmd.stdout }}"

- name: Worker setup
  hosts: workers
  become: true
  tasks:
    - name: Join the cluster
      ansible.builtin.command: "{{ hostvars[groups['master'][0]].join_command
}}"
      args:
        creates: /etc/kubernetes/kubelet.conf

- name: Verify the cluster
  hosts: master
  become: true
  tasks:
```

```
    - name: Wait until all nodes are Ready
      ansible.builtin.command: kubectl wait --for=condition=Ready nodes --all
--timeout=300s
      environment:
        KUBECONFIG: /etc/kubernetes/admin.conf
      changed_when: false

    - name: List nodes
      ansible.builtin.command: kubectl get nodes -o wide
      environment:
        KUBECONFIG: /etc/kubernetes/admin.conf
      register: nodes_out
      changed_when: false

    - name: Show nodes
      ansible.builtin.debug:
        var: nodes_out.stdout_lines
```